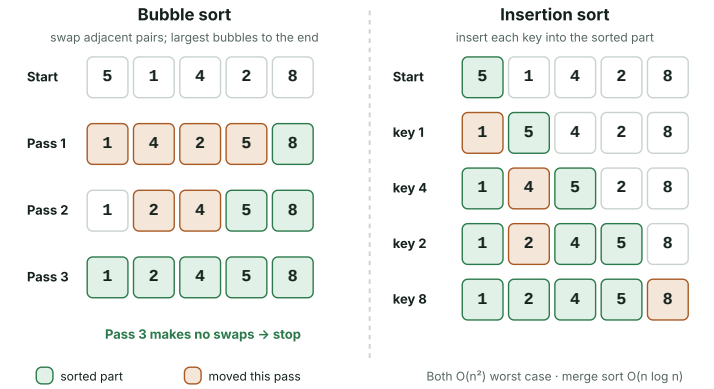
Sorting algorithms

AP18 FACT FILE

Specification: students should be able to

- Describe, trace and implement bubble sort and insertion sort using pseudocode and C#
- Describe the principles of merge sort and quick sort as divide-and-conquer algorithms
- Compare sorting algorithms in terms of time efficiency, memory use and suitability for a given data set
- Explain how design features such as a swapped flag improve the efficiency of an algorithm

Key words: bubble sort, insertion sort, merge sort, quick sort, pass, adjacent, swap, temporary variable, swapped flag, key, nested loop, divide and conquer, pivot, partition, in place, $O(n^2)$, $O(n \log n)$



Bubble sort and insertion sort of 5 1 4 2 8 shown pass by pass side by side

Sorting: the big picture

Sorting arranges data into ascending or descending order, numerically or alphabetically. It is one of the most common tasks in computing because sorted data is easier for people to read, as in a league table or class list, and easier for programs to process: a sorted list can be searched with a fast binary search, and duplicates, the largest value or the median can be found quickly. The AS course requires four sorting algorithms. Bubble sort and insertion sort are simple algorithms built from nested loops; they are easy to understand, trace and code, and sort the data in place, but the number of comparisons grows with the square of the number of items, so they become very slow for large lists. Merge sort and quick sort use a divide-and-conquer approach, repeatedly breaking the list into smaller parts and combining the results; they are harder to code, usually using recursion, but for large lists they are dramatically faster. Choosing between them means weighing time efficiency, memory use, the size and state of the data and how easy the code is to write and maintain.

Bubble sort

Bubble sort works through the list comparing each pair of adjacent items and swapping them if they are in the wrong order. One complete run through the list is called a pass. In an ascending sort the largest value is carried to the end of the list during the first pass, so it is in its final position, and each later pass can stop one item earlier. Swapping two array elements requires a temporary variable: the first value is copied to temp, the second is copied into the first position and temp is copied into the second; without temp one of the values would be overwritten and lost. A Boolean flag, often called swapped, is set to false at the start of each pass and to true whenever a swap is made. If a whole pass makes no swaps the list must be sorted, so the algorithm stops. This means an already sorted list needs only one pass of $n - 1$ comparisons, giving a best case of $O(n)$. In the worst case, a list in reverse order, about $n^2/2$ comparisons are needed, so the average and worst cases are $O(n^2)$.

Insertion sort

Insertion sort builds a sorted section at the start of the list, much as a card player arranges a hand of cards one at a time. Initially the first item on its own is treated as sorted. On each pass the next unsorted item is copied into a variable called the key. Working from right to left through the sorted section, every item larger than the key is shifted one place to the right, and when a smaller item or the start of the list is reached, the key is inserted into the gap. The sorted section therefore grows by one item per pass, and a list of n items needs n − 1 passes. Insertion sort sorts in place, needing only one extra variable. If the list is already sorted, each key is compared once and nothing moves, so the best case is O(n); if the list is in reverse order every key must move to the front, giving a worst case of $O(n^2)$. It usually makes fewer comparisons than bubble sort and is very effective for small lists, nearly sorted lists and adding a few new items to a list that is already in order.

Tracing a sort step by step

Sorting questions often ask you to show the state of a list after each pass, so careful tracing is essential. For a bubble sort, work along the list one adjacent pair at a time, making each swap before moving to the next pair; a common error is to rearrange several items at once. For the netball points 14, 9, 22, 5, 17, 11, the first pass swaps 14 and 9, leaves 14 and 22, then carries 22 past 5, 17 and 11 to give 9, 14, 5, 17, 11, 22. The second pass gives 9, 5, 14, 11, 17, 22 and the third gives 5, 9, 11, 14, 17, 22. A fourth pass makes no swaps, so the sort stops after 14 comparisons. For an insertion sort of the same list, record the key on each pass and how many items shift: 9 moves before 14, 22 stays put, 5 moves to the front past three items, 17 moves past 22, and 11 moves past 22, 17 and 14. Both traces end with the same sorted list, but the intermediate lists are different, which is how an examiner can tell which algorithm was applied.

Divide and conquer: merge sort

Merge sort repeatedly divides a list in half until every sub-list contains a single item, which is sorted by definition. It then merges the sub-lists back together in pairs. Merging two sorted lists is straightforward: compare the front items of each, move the smaller one into the new list, and repeat until one list is empty, then copy across whatever remains of the other. For example, 6, 2, 9, 4, 7, 1, 8, 3 is divided into eight single items, merged into the pairs [2 6], [4 9], [1 7] and [3 8], then into [2 4 6 9] and [1 3 7 8], and finally into 1, 2, 3, 4, 6, 7, 8, 9. Splitting a list of n items in half repeatedly produces about $\log_2 n$ levels, and merging at each level handles all n items, so merge sort takes O(n log n) time in the best, average and worst cases. This guaranteed performance makes it predictable for large data sets. Its main drawback is memory: the sub-lists must be stored while merging, so it needs extra space roughly equal to the size of the list.

Divide and conquer: quick sort

Quick sort chooses one item, the pivot, and partitions the rest of the list so that items smaller than the pivot are placed to its left and items larger than it to its right. The pivot is then in its final sorted position. The same process is applied recursively to the left and right sub-lists until every sub-list has zero or one item. Using the first item as the pivot, 6, 2, 9, 4, 7, 1, 8, 3 partitions around 6 into [2 4 1 3] and [9 7 8]; the left part partitions around 2 into [1] and [4 3], and so on until the whole list is sorted. When the pivot divides each list roughly in half, quick sort takes O(n log n) time and, because partitioning can be done within the original array, it needs little extra memory; in practice it is often the fastest general-purpose sort. However, if the pivot is consistently the smallest or largest item, for example choosing the first item of an already sorted list, each partition removes only one item and the time degrades to $O(n^2)$. Choosing the middle item, or a random item, as the pivot reduces this risk.

Key terms from the mark schemes

Bubble sort
A sort that repeatedly compares adjacent items and swaps them if they are in the wrong order until a pass makes no swaps.

Insertion sort
A sort that takes each item in turn and inserts it into its correct position within an already sorted section of the list.

Merge sort
A divide-and-conquer sort that splits a list into single items and then merges sub-lists back together in order.

Quick sort
A divide-and-conquer sort that partitions items around a pivot and then recursively sorts each partition.

Divide and conquer
An approach that solves a problem by repeatedly breaking it into smaller sub-problems of the same type and combining their solutions.

Pass
One complete traversal through the list (or the unsorted part of it) by a sorting algorithm.

Pivot
The item chosen in quick sort around which the other items are partitioned into smaller and larger groups.

Swapped flag
A Boolean variable in a bubble sort that records whether any swap was made during a pass, so the sort can stop early.
Credited: Boolean flag

Temporary variable
A variable used to hold one value during a swap so that it is not overwritten and lost.
Credited: temp

In place
Describes a sort that rearranges items within the original array, needing little or no extra memory.

C C# example

Bubble sort in C#

```
int[] points = { 14, 9, 22, 5, 17, 11 };
bool swapped;
int end = points.Length - 1;
do
{
    swapped = false;
    for (int i = 0; i < end; i++)
    {
        if (points[i] > points[i + 1])
        {
            int temp = points[i];
            points[i] = points[i + 1];
            points[i + 1] = temp;
            swapped = true;
        }
    }
    end--;
} while (swapped);
Console.WriteLine(string.Join(" ", points));   // 5 9 11 14 17 22
```

Exam focus

For trace questions, show the list after every pass, not just the final result, and follow the algorithm exactly as given; if asked for a specific pass, count carefully, since the first pass is often already shown. When describing bubble sort, credited points include comparing adjacent items, swapping them if they are in the wrong order, moving along to the end of the list, repeating passes and stopping when a pass makes no swaps. For insertion sort, mention taking each item in turn, comparing it with the sorted part, shifting larger items and inserting it in the correct place. Merge sort descriptions need both the splitting into single items and the merging of sorted sub-lists; quick sort descriptions need the pivot, the partition and recursion on each side. Explain questions about the temporary variable or the swapped flag carry two marks: what it does and why it matters. For 'compare' or 'evaluate' questions, use time efficiency for large n, memory use, behaviour on nearly sorted data and ease of coding, and finish with a justified choice for the scenario rather than a list of facts.

? Check yourself

1. Show the list 6, 3, 8, 2 after each pass of a bubble sort into ascending order. [3]
2. Show the list 6, 3, 8, 2 after each pass of an insertion sort into ascending order. [3]
3. Explain why a temporary variable is needed when swapping two items in an array. [2]

Answers: 1. Pass 1: 3 6 2 8 (1) / Pass 2: 3 2 6 8 (1) / Pass 3: 2 3 6 8 (1) 2. Pass 1: 3 6 8 2 (1) / Pass 2: 3 6 8 2 (1) / Pass 3: 2 3 6 8 (1) 3. When the first value is overwritten by the second, the original first value would be lost (1) / so it is copied to a temporary variable first and then copied into the second position (1)